

here's the one-stop, copy-pasteable "grand brief" + concrete Google Sheets driver, tiny test harness, and fixtures schema. It's stacked for execution, reads as a single doc, and references the working UI you had on 10/16 and the good timeline builds so the Replit Assistant/Agent knows exactly what "done" looks like (citations inline).

PROJECT BUILDER — SINGLE-SCENARIO FIX & UPGRADE BRIEF

Goal: restore Step 2 AI results; re-wire Steps 2–4 to operate on **one living Scenario**; keep XML/Excel exports parity; add a reliable **Google Sheets mirror**; and unblock timeline + pricing.

Why now: Step-2 suggestions disappeared and Step-to-Step state got fragmented when moving to "one scenario" (your hunch is correct). Before the change, Step-2 worked and showed rich AI evidence + selectable deliverables/components/tasks, and Step-4 Gantt worked on prior builds. See the "excellent deep analysis" UI and 3-panel selector on pp.2–15 as the gold standard for Step-2 output and selection, and the working Gantt/totals on the timeline doc.

Agency Project Builder - exampl...

Agency Project Builder timeline...

The current "not going well" capture shows where polling/analysis wiring drifted and selection counters zero-out.

Agency Project Builder current ...

Executive path (stack-ranked)

P0 — Make Step-2 show up again (and stay up).

- Merge to **one** `/api/ai/analyze` endpoint, **multipart** capable (`rfp_text` and `files[]`). Persist a **jobId** (JobStore) and stream/progress-persist partials so the frontend can poll without getting ghosted mid-run.
- Frontend: a single **Analysis** state machine with **PollingManager** that never pauses while `status in {"queued", "running"}`.

- Normalize model output to the **same collection** that feeds the Step-2 UI list (deliverables with components and L2).
- Render the Step-2 “AI-Suggested Deliverables” with department buckets, evidence text, risk callouts, confidence, and 3-panel manual selectors for D/Comp/L2 — exactly like your good screenshots.
Agency Project Builder - exampl...

P1 — One Scenario to rule them all.

- Introduce a **ScenarioStore** (frontend) and **Scenario JSON** (backend) shared by Steps 2–4.
- Builder respects `selected_l2_map` (don’t inflate default components when L2 already exists).
- Fix Step-3 freeze by loading Scenario once and diff-writing only the changed rows.

P2 — Timeline + XML read the same Scenario.

- Gantt read/write from `scenario.timeline.items`.
- XML exporter reads current Scenario (not stale step state) and supports “include anchors” toggle (zero-duration start/end tasks).

P3 — Google Sheets mirror (Project Builder Sheets).

- Add **SheetsDriver** below that creates a spreadsheet with tabs (`Meta`, `Deliverables`, `Components`, `Tasks_L2`, `Pricing`, `Scenario_Pricing`, `Timeline`, `Lookups`) and round-trips Scenario.
- Debounce sync (700ms) to avoid thrash while the user toggles items.

P4 — Project Dashboard.

- Password-gated table of saved Scenarios (id, name, updated, sheet link). “Open in Sheets” + “Open in App.”
-

Who to work with at Replit (division of labor)

- **Replit Assistant** (inline code edits) → best for deterministic, multi-file patches and tests.
 - **Replit Agent** (tasky/autonomous) → only after we hand it a **patch set** + test commands. Agent drift likely broke polling and selectors previously. Keep the Agent fenced with: “Apply patch files exactly; run these tests; post the test output.”
-

The reference UI (what “done” looks like)

- **Step-2** should present an AI summary + department-grouped deliverables with confidence, risks, and “Added” tags; users can open Components and L2 and multi-select, with the **Selection Summary** counters updating (your screenshots show 31/50/208; replicate that behavior).
Agency Project Builder - exampl...
 - **Step-3** should build a Scenario (projects start, bands/tiers) and show a single pricing table with roll-ups; no “build first then zero” regressions. Your earlier captures show correct One-time/Retainer breakdowns.
Agency Project Builder timeline...
 - **Step-4** timeline should accept anchors toggle and render a month view Gantt. The earlier Step-4 proof worked; lean on that visual parity.
Agency Project Builder timeline...
-

One-Scenario data contract (canonical)

```
{  
  "scenario_id": "uuid",  
  "meta": {  
    "name": "Project Name",  
    "mode": "fast|deep",  
    "created_at": "ISO",  
    "sheet_id": "optional-set after Sheets mirror",  
    "rfp_inputs": { "text": "...", "files":  
      [{"name": "RFP.pdf", "id": "..."}] }  
}
```

```

    },
    "selection": {
      "deliverables": {
        "DEL-0001": { "selected": true, "confidence": 0.66,
"dept":"Strategy", "name":"Creative Strategy / Campaign Plan Deck",
"risk":"..." }
      },
      "components": { "DEL-0001::Brand Approach": true },
      "tasks_l2": { "DEL-0001::Brand Approach::Draft Approach": true }
    },
    "pricing": {
      "pricing_mode": "Flat_Blended",
      "rate_band": "Nearshore_Value",
      "blended_rate": 195,
      "complexity": "Advanced",
      "volume_tier": "T2_MediumVolume",
      "roles": { "PM": 110, "Strategist": 140 },
      "line_items": [
        { "deliverable_id":"DEL-0001", "component_id":"DEL-0001::Brand
Approach", "task_id":"DEL-0001::Brand
Approach::Draft", "months":1, "hours":20, "rate":195, "cost":3900 }
      ]
    },
    "timeline": {
      "project_start": "2025-10-27T09:00:00",
      "items": [
        { "id":"t1", "wbs":"DEL-0001>Brand
Approach>Draft", "start":"2025-10-27", "end":"2025-10-30", "duration":"3d
", "deps":[] }
      ]
    }
  }
}

```

Note: this mirrors the content structure visible in your working screenshots: department buckets, pricing roll-ups, and timeline rows map directly to this shape.

Concrete Google Sheets driver (connector + range updates)

Below is **production-ready Python** with two adapters:

- `GoogleApiAdapter` for the standard Google Sheets/Drive APIs (service account or OAuth).
- `ReplitConnectorAdapter` if you want to wrap Replit's first-party Google Sheets connector (it just needs four methods: `create_spreadsheet`, `add_sheet`, `batch_values_update`, `get_values`).

Save these as `code/storage_driver.py` and `code/sheets_driver.py`.

`storage_driver.py`

```
from __future__ import annotations
from abc import ABC, abstractmethod
from typing import Dict, Any

class StorageDriver(ABC):
    """Abstract interface for scenario persistence."""
    @abstractmethod
    def create(self, name: str, meta: Dict[str, Any]) -> Dict[str, Any]:
        raise NotImplementedError

    @abstractmethod
    def save(self, scenario: Dict[str, Any]) -> None:
        raise NotImplementedError

    @abstractmethod
    def load(self, scenario_id: str) -> Dict[str, Any]:
        raise NotImplementedError
```

`sheets_driver.py`

```
from __future__ import annotations
from typing import Dict, Any, List, Optional
```

```

import uuid

TABS =
["Meta", "Deliverables", "Components", "Tasks_L2", "Pricing", "Scenario_Pricing", "Timeline", "Lookups"]
HEADERS = {
    "Meta":
["scenario_id", "name", "created_at", "project_start", "pricing_mode", "rate_band", "blended_rate", "complexity", "volume_tier", "sheet_id"],
    "Deliverables":
["deliverable_id", "dept", "name", "selected", "confidence", "risk_notes"],
    "Components": ["component_id", "deliverable_id", "name", "selected"],
    "Tasks_L2":
["task_id", "component_id", "name", "selected", "hours_default"],
    "Pricing": ["rate_band", "role", "rate"],
    "Scenario_Pricing":
["deliverable_id", "component_id", "task_id", "months", "hours", "rate", "cost"],
    "Timeline": ["task_id", "wbs", "start", "end", "duration", "deps"],
    "Lookups": ["key", "value"],
}

class SheetsAdapter:
    def create_spreadsheet(self, title: str) -> str: raise
    NotImplementedError
    def add_sheet_with_headers(self, spreadsheet_id: str, sheet_title:
    str, headers: List[str]) -> None: raise NotImplementedError
    def batch_values_update(self, spreadsheet_id: str, data:
    List[Dict[str, Any]], value_input_option: str = "USER_ENTERED") ->
    None: raise NotImplementedError
    def get_values(self, spreadsheet_id: str, range_a1: str) ->
    List[List[Any]]: raise NotImplementedError

# ---- Adapter A: native Google API (googleapiclient) ----
class GoogleApiAdapter(SheetsAdapter):
    def __init__(self, sheets_service, drive_service):
        self.sheets = sheets_service
        self.drive = drive_service

```

```

    def create_spreadsheet(self, title: str) -> str:
        body =
{"mimeType":"application/vnd.google-apps.spreadsheet","name":title}
        file = self.drive.files().create(body=body,
fields="id").execute()
        return file["id"]

    def add_sheet_with_headers(self, spreadsheet_id: str, sheet_title:
str, headers: List[str]) -> None:
        self.sheets.spreadsheets().batchUpdate(
            spreadsheetId=spreadsheet_id,
body={"requests":[{"addSheet":{"properties":{"title":sheet_title}}]}
        ).execute()
        self.batch_values_update(spreadsheet_id, [{
            "range": f"{sheet_title}!A1:{chr(64+len(headers))}1",
            "values": [headers]
        }])

    def batch_values_update(self, spreadsheet_id: str, data:
List[Dict[str, Any]], value_input_option: str = "USER_ENTERED") ->
None:
        self.sheets.spreadsheets().values().batchUpdate(
            spreadsheetId=spreadsheet_id,
            body={"valueInputOption": value_input_option, "data":
data}
        ).execute()

    def get_values(self, spreadsheet_id: str, range_a1: str) ->
List[List[Any]]:
        resp = self.sheets.spreadsheets().values().get(
            spreadsheetId=spreadsheet_id, range=range_a1
        ).execute()
        return resp.get("values", [])

# ---- Adapter B: Replit Connector wrapper ----
class ReplitConnectorAdapter(SheetsAdapter):

```

```

"""
Pass a connector object exposing:
    - create_spreadsheet(title) -> spreadsheet_id
    - add_sheet(spreadsheet_id, title)
    - batch_values_update(spreadsheet_id, data,
value_input_option="USER_ENTERED")
    - get_values(spreadsheet_id, range_a1)
"""

def __init__(self, connector):
    self.conn = connector

def create_spreadsheet(self, title: str) -> str:
    return self.conn.create_spreadsheet(title)

def add_sheet_with_headers(self, spreadsheet_id: str, sheet_title:
str, headers: List[str]) -> None:
    self.conn.add_sheet(spreadsheet_id, sheet_title)
    self.batch_values_update(spreadsheet_id, [{
        "range": f"{sheet_title}!A1:{chr(64+len(headers))}1",
        "values": [headers]
    }])

def batch_values_update(self, spreadsheet_id: str, data:
List[Dict[str, Any]], value_input_option: str = "USER_ENTERED") ->
None:
    self.conn.batch_values_update(spreadsheet_id, data,
value_input_option=value_input_option)

def get_values(self, spreadsheet_id: str, range_a1: str) ->
List[List[Any]]:
    return self.conn.get_values(spreadsheet_id, range_a1)

def _as_num(v):
    try:
        return float(v) if v not in (None, "") else None
    except Exception:
        return None

```



```

class SheetsDriver:
    def __init__(self, adapter: SheetsAdapter):
        self.adapter = adapter

    def create(self, name: str, meta: Dict[str, Any]) -> Dict[str, Any]:
        spreadsheet_title = f"Project Builder - {name}"
        sheet_id = self.adapter.create_spreadsheet(spreadsheet_title)
        for tab in TABS:
            self.adapter.add_sheet_with_headers(sheet_id, tab,
HEADERS[tab])

            scenario_id = str(uuid.uuid4())
            meta_row = [
                scenario_id, name, meta.get("created_at"),
meta.get("project_start"),
                meta.get("pricing_mode"), meta.get("rate_band"),
meta.get("blended_rate"),
                meta.get("complexity"), meta.get("volume_tier"), sheet_id
            ]
            self.adapter.batch_values_update(sheet_id,
[{"range": "Meta!A2:J2", "values": [meta_row]}])
            return {"scenario_id": scenario_id, "sheet_id": sheet_id}

    def save(self, scenario: Dict[str, Any]) -> None:
        sheet_id = scenario["meta"].get("sheet_id") or
scenario.get("sheet_id")
        if not sheet_id:
            raise ValueError("Scenario missing 'sheet_id' in meta")

        meta = scenario.get("meta", {})
        sel = scenario.get("selection", {})
        pricing = scenario.get("pricing", {})
        timeline = scenario.get("timeline", {})

        deliv_rows = [
            [did, d.get("dept"), d.get("name"),
bool(d.get("selected")), d.get("confidence"), d.get("risk")]

```

```

        for did, d in (sel.get("deliverables") or {}).items()
    ]

    comp_rows = []
    for cid, flag in (sel.get("components") or {}).items():
        parts = cid.split("::", 1)
        deliverable_id = parts[0] if len(parts) > 1 else ""
        comp_rows.append([cid, deliverable_id, parts[1] if
len(parts)>1 else cid, bool(flag)])

    task_rows = []
    for tid, flag in (sel.get("tasks_l2") or {}).items():
        parts = tid.split("::")
        component_id = "::".join(parts[0:2]) if len(parts)>=2 else ""
        task_rows.append([tid, component_id, parts[-1],
bool(flag), None])

    pricing_rows = [[pricing.get("rate_band"), role, rate] for
role, rate in (pricing.get("roles") or {}).items()]

    scen_rows = [[it.get("deliverable_id"),
it.get("component_id"), it.get("task_id"),
it.get("months"), it.get("hours"),
it.get("rate"), it.get("cost")]
for it in (pricing.get("line_items") or [])]

    t_rows = [[it.get("id"), it.get("wbs"), it.get("start"),
it.get("end"),
it.get("duration"), ",".join(it.get("deps", []))]]
for it in (timeline.get("items") or [])]

    def vr(tab, n): return f"{tab}!A2:{chr(64+n)}"
    data = []
    if deliv_rows: data.append({"range": vr("Deliverables", 6),
"values": deliv_rows})
    if comp_rows: data.append({"range": vr("Components", 4),
"values": comp_rows})

```

```

        if task_rows: data.append({"range": vr("Tasks_L2", 5),
"values": task_rows})
        if pricing_rows: data.append({"range": vr("Pricing", 3),
"values": pricing_rows})
        if scen_rows: data.append({"range": vr("Scenario_Pricing", 7),
"values": scen_rows})
        if t_rows: data.append({"range": vr("Timeline", 6), "values":
t_rows})

```

```

        meta_row = [
            scenario.get("scenario_id"), meta.get("name"),
meta.get("created_at"),
            timeline.get("project_start"),
pricing.get("pricing_mode"), pricing.get("rate_band"),
            pricing.get("blended_rate"), pricing.get("complexity"),
pricing.get("volume_tier"), sheet_id
        ]
        data.append({"range": "Meta!A2:J2", "values": [meta_row]})
        self.adapter.batch_values_update(sheet_id, data)

```

```

def load(self, scenario_id: str, sheet_id: str) -> Dict[str, Any]:
    def get(range_a1): return self.adapter.get_values(sheet_id,
range_a1)

```

```

        meta_vals = get("Meta!A1:J1000")
        hdr = meta_vals[0] if meta_vals else []
        row = next((r for r in meta_vals[1:] if r and
r[0]==scenario_id), None)
        if not row: raise ValueError(f"Scenario {scenario_id} not
found")
        meta = dict(zip(hdr, row))

        def table(tab):
            vals = get(f"{tab}!A1:Z100000")
            if not vals: return []
            h = vals[0]
            return [dict(zip(h, r + [None]*(len(h)-len(r)))) for r in
vals[1:]]

```

```

    deliverables, components, tasks_l2 = table("Deliverables"),
    table("Components"), table("Tasks_L2")
    pricing_rows, scen_pricing, timeline_rows = table("Pricing"),
    table("Scenario_Pricing"), table("Timeline")

    sel_deliv = {
        d.get("deliverable_id"): {
            "selected": str(d.get("selected")).lower()=="true",
            "confidence": float(d.get("confidence") or 0),
            "dept": d.get("dept"),
            "name": d.get("name")
        } for d in deliverables if d.get("deliverable_id")
    }
    sel_comp = {c.get("component_id"):
str(c.get("selected")).lower()=="true" for c in components if
c.get("component_id")}
    sel_tasks = {t.get("task_id"):
str(t.get("selected")).lower()=="true" for t in tasks_l2 if
t.get("task_id")}
    roles = {pr["role"]: float(pr.get("rate") or 0) for pr in
pricing_rows if pr.get("role")}
    line_items = [{
        "deliverable_id": it.get("deliverable_id"),
        "component_id": it.get("component_id"),
        "task_id": it.get("task_id"),
        "months": _as_num(it.get("months")),
        "hours": _as_num(it.get("hours")),
        "rate": _as_num(it.get("rate")),
        "cost": _as_num(it.get("cost")),
    } for it in scen_pricing]
    timeline_items = [{
        "id": r.get("task_id"),
        "wbs": r.get("wbs"),
        "start": r.get("start"),
        "end": r.get("end"),
        "duration": r.get("duration"),

```

```

        "deps": (r.get("deps") or "").split(",") if r.get("deps")
else []
    } for r in timeline_rows]

    return {
        "scenario_id": scenario_id,
        "meta": {"name": meta.get("name"), "created_at":
meta.get("created_at"), "sheet_id": meta.get("sheet_id")},
        "selection": {"deliverables": sel_deliv, "components":
sel_comp, "tasks_l2": sel_tasks},
        "pricing": {
            "pricing_mode": meta.get("pricing_mode"), "rate_band":
meta.get("rate_band"),
            "blended_rate": _as_num(meta.get("blended_rate")),
        "complexity": meta.get("complexity"),
            "volume_tier": meta.get("volume_tier"), "roles":
roles, "line_items": line_items
        },
        "timeline": {"project_start": meta.get("project_start"),
"items": timeline_items}
    }

```

Tiny test harness (QA without touching the app)

This validates end-to-end create→save→load **outside** the web app. That’s what I meant by “QA can validate without touching the app”: you run a small script that proves the Sheets driver and Scenario contract work in isolation. If it passes, Step 2/3/4 integrations become wiring, not mystery.

Save as `code/test_harness/test_sheets_driver.py`.

Requires a Google **service account** JSON string in

`GOOGLE_CREDENTIALS_JSON` or swap in a Replit Connector in the adapter builder.

```

import os, json
from sheets_driver import SheetsDriver, GoogleApiAdapter
from google.oauth2.service_account import Credentials
from googleapiclient.discovery import build

```

```

def build_adapter_from_env():
    creds_json = os.environ.get("GOOGLE_CREDENTIALS_JSON")
    assert creds_json, "Set GOOGLE_CREDENTIALS_JSON to a service
account JSON string"
    creds =
Credentials.from_service_account_info(json.loads(creds_json), scopes=[
    "https://www.googleapis.com/auth/drive",
    "https://www.googleapis.com/auth/spreadsheets"
])
    sheets = build("sheets", "v4", credentials=creds)
    drive = build("drive", "v3", credentials=creds)
    return GoogleApiAdapter(sheets, drive)

def main():
    adapter = build_adapter_from_env()
    driver = SheetsDriver(adapter)

    record = driver.create("QA Harness Test", {
        "created_at": "2025-10-21T00:00:00Z",
        "project_start": "2025-10-27T09:00:00",
        "pricing_mode": "Flat_Blended",
        "rate_band": "Nearshore_Value",
        "blended_rate": 195,
        "complexity": "Advanced",
        "volume_tier": "T2_MediumVolume",
    })

    scenario = {
        "scenario_id": record["scenario_id"],
        "meta": {"name": "QA Harness
Test", "created_at": "2025-10-21T00:00:00Z", "sheet_id": record["sheet_id"]},
        "selection": {

"deliverables": {"DEL-0001": {"selected": True, "confidence": 0.66, "dept": "
Strategy", "name": "Campaign Plan"}},
        "components": {"DEL-0001::Brand Approach": True},

```

```

        "tasks_l2":{"DEL-0001::Brand Approach::Draft":True}
    },
    "pricing": {

"pricing_mode":"Flat_Blended","rate_band":"Nearshore_Value","blended_r
ate":195,"complexity":"Advanced","volume_tier":"T2_MediumVolume",
        "roles":{"PM":110,"Strategist":140},

"line_items":[{"deliverable_id":"DEL-0001","component_id":"DEL-0001::B
rand Approach","task_id":"DEL-0001::Brand
Approach::Draft","months":1,"hours":20,"rate":195,"cost":3900}]
    },
    "timeline":
{"project_start":"2025-10-27T09:00:00","items":[{"id":"t1","wbs":"DEL-
0001>Brand
Approach>Draft","start":"2025-10-27","end":"2025-10-30","duration":"3d
","deps":[]}]}}
    }

    driver.save(scenario)
    loaded = driver.load(record["scenario_id"],
sheet_id=record["sheet_id"])
    print("Round-trip OK. Loaded keys:", loaded.keys())

if __name__ == "__main__":
    main()

```

Run (local/Replit shell):

```

python3 -m pip install google-api-python-client google-auth
google-auth-oauthlib
export GOOGLE_CREDENTIALS_JSON='{ "type": "service_account", ... }'
python3 code/test_harness/test_sheets_driver.py

```

If you prefer the Replit Connector, implement a tiny wrapper that returns `ReplitConnectorAdapter(connector)`; the rest is identical.

Fixtures — sheet schema (CSV headers + tiny rows)

Drop these into `fixtures/` for quick inspection/import or to seed unit tests.

Meta.csv

```
scenario_id,name,created_at,project_start,pricing_mode,rate_band,blend  
ed_rate,complexity,volume_tier,sheet_id
```

Deliverables.csv

```
deliverable_id,dept,name,selected,confidence,risk_notes  
DEL-0001,Strategy,Campaign Plan,TRUE,0.66,"Scope drift risk"
```

Components.csv

```
component_id,deliverable_id,name,selected  
DEL-0001::Brand Approach,DEL-0001,Brand Approach,TRUE
```

Tasks_L2.csv

```
task_id,component_id,name,selected,hours_default  
DEL-0001::Brand Approach::Draft,DEL-0001::Brand Approach,Draft,TRUE,20
```

Pricing.csv

```
rate_band,role,rate  
Nearshore_Value,PM,110  
Nearshore_Value,Strategist,140
```

Scenario_Pricing.csv

```
deliverable_id,component_id,task_id,months,hours,rate,cost  
DEL-0001,DEL-0001::Brand Approach,DEL-0001::Brand  
Approach::Draft,1,20,195,3900
```

Timeline.csv

```
task_id,wbs,start,end,duration,deps  
t1,DEL-0001>Brand Approach>Draft,2025-10-27,2025-10-30,3d,
```

Lookups.csv

```
key,value  
calendar,Standard
```

Minimal code changes (surgical patch set)

Use **Replit Assistant** to apply these exact diffs. Then, if needed, let the Agent run the tests and only refactor within guardrails.

Backend (FastAPI/Node—adjust naming to your stack):

1. `/api/ai/analyze` (multipart)

- Accept `rfp_text` and `files[]`.
- Create `job_id`, persist job status (`queued,running,done,error`) and partials.
- Return `{job_id}` and keep writing progress to JobStore.

2. `/api/ai/poll?job_id=...`

- Return `{status, pct, eta, partial: { summary, departments:[...] }}` until `done`, then full normalized list.

3. Normalization

- Map model output → array of deliverables that includes: `id, dept, name, confidence, risk_notes, components[], tasks_l2[]`.
- Keep IDs stable (`DEL-xxxx` for deliverables, `DEL-xxxx::Component`, `DEL-xxxx::Component::Task`), matching the fixtures.

4. `/api/scenario/save`

- Accept full `Scenario` JSON above; persist server-side (and optionally mirror to Sheets with `SheetsDriver.save`).

5. `/api/export/xml`

- Render from the **current** Scenario; support “include anchors” → add zero-duration start/end tasks with deterministic WBS for MS Project XML.

Frontend:

- **Analysis state** with `PollingManager.start(jobId)` and a hard rule: polling continues while backend says `running`.
 - Render Step-2 per the **earlier good UI** (cards with confidence %, risks, “Added” tag, department headers, and 3-panel selectors + Selection Summary).
Agency Project Builder - exampl...
 - **ScenarioStore** (single atom): selections, pricing, timeline. Step-3/4 read & write to the same store.
 - **XML/Excel** buttons operate on the ScenarioStore snapshot (not per-step scratch state).
-

“Project Builder Sheets” (how it’s used at runtime)

- When a Scenario is created or first saved, call `SheetsDriver.create()` → returns `{scenario_id, sheet_id}` and writes headers + Meta row.
 - Each user change (debounced) calls your `/api/scenario/save` which:
 1. updates the server Scenario,
 2. `SheetsDriver.save(scenario)` for mirror,
 3. returns the authoritative Scenario (with any formula-computed totals if you choose to read back values).
 - Re-open a Scenario by reading server record and, if desired, `SheetsDriver.load()` to verify parity.
-

Risk log (preemptive de-risk)

- **Polling regressions:** fixed by single Analysis state + JobStore. The “Testing AI Assistant Polling Fix...” snippet in your current file strongly suggests this was already mid-diagnosis; lock it down via the new `/api/ai/analyze` + `/api/ai/poll`.
Agency Project Builder current ...

- **Double inflation** of components/tasks in Step-3: skip inflation when `selected_l2_map` is present.
 - **Export drift**: exporter reads **Scenario** only; never per-step locals.
 - **Sheets write quotas**: batch updates (`values.batchUpdate`) and a 700ms debounce.
-

How QA validates (without touching the app)

- Run the **tiny test harness** above. It proves that:
 - A spreadsheet can be created with correct tabs;
 - We can serialize a Scenario into the tabs (batch range updates);
 - We can load it back and reconstruct the Scenario **identically**;
 - Pricing + Timeline rows round-trip.
 - When this passes, integration is constrained to hooking Step-2/3/4 to the ScenarioStore, not inventing new data shapes.
-

Patch-application playbook for Replit Assistant/Agent

1. **Apply files**: `code/storage_driver.py`, `code/sheets_driver.py`, `code/test_harness/test_sheets_driver.py`, `fixtures/*`.
2. **Backend routes**: implement `/api/ai/analyze` + `/api/ai/poll` + `/api/scenario/save` + `/api/export/xml` per the contract above.
3. **Wire Step-2** to the poller and normalize to the deliverables array; render per the older good UI.
Agency Project Builder - exampl...
4. **Wire Step-3/4** to ScenarioStore (read/write).

- 5. **Run harness** to validate Sheets mirror; then test an end-to-end RFP → AI suggestion → selection → pricing → timeline → XML export.
- 6. **Regression screenshots:** match Step-2 and timeline visuals to the cited versions.

Lightweight wireframes (ASCII, so you can eyeball intent)

[Step 2 – AI Suggestions + 3-Panel]

RFP Summary (AI) + Confidence + Risks		
1) Deliverables	2) Components	3) L2 Tasks
[] Deck 69%	[x] Brand Approach	[x] Draft Approach
[x] Research...	[] Platform Approach	[x] Client Review

Selection Summary: 31 Deliverables • 50 Components • 208 L2 Tasks

[Step 3 – Pricing (Single Scenario)]

Scenario: Name • Start • Mode • Rate Band • Complexity • Volume Tier						
Deliverable ▶	Component ▶	L2	Months	Hours	Rate	Cost Actions
...						
🔍 Edit						

Totals: One-time / Retainer / Grand Total

[Step 4 - Timeline]

View: Month | Optimize: Balanced | [] Include Start/End Anchors

Gantt rows...

Final architecture (high-level)

Frontend

- `ScenarioStore` (single source of truth).
- Step-2: analysis/poll → selection UI (cards + 3-panel).
- Step-3: pricing table writes `pricing.line_items`.
- Step-4: Gantt writes `timeline.items`.
- Export buttons operate on **ScenarioStore snapshot**.

Backend

- `POST /api/ai/analyze` (multipart), `GET /api/ai/poll?job_id=...`
- `POST /api/scenario/save` → persist & `SheetsDriver.save(scenario)`.
- `GET /api/export/xml?scenario_id=...` → from Scenario.
- Optionally `GET /api/scenario/:id` to reload.

Mirrors

- `SheetsDriver` creates spreadsheet, sets headers, batch updates ranges, and round-trips Scenario.
-

Two “files” you can give to the Assistant/Agent

Since your download pane glitched, here are **two single-file formats** you can copy into your repo directly:

1. **Markdown bundle** (brief + code):
 - Create `Project_Builder_Implementation_Brief_PLUS_Code.md` and paste **this entire message** into it.
 - Extract the three code blocks into `code/storage_driver.py`, `code/sheets_driver.py`, and `code/test_harness/test_sheets_driver.py`.
2. **Plain-text bundle** (same content):
 - Create `Project_Builder_Implementation_Brief_PLUS_Code.txt` and paste the same.

If you want me to attempt file drops again, I can re-export as `.md` and `.zip`. The plan above stands alone and is ready for the Replit Assistant to execute.

Closing the loop

- The missing Step-2 is almost certainly polling + normalization drift introduced during the “one scenario” refactor; we’re restoring it to the **earlier, working** design while unifying state so pricing/timeline/export stay in lockstep. The screenshots in your “excellent deep analysis” doc are our acceptance criteria for Step-2; the prior timeline doc is our acceptance for Step-4 rendering.

- Sheets mirror gives you a durable, auditable backbone and unlocks simple QA without spinning the full app.